CSC 370: Databases Boyce-Codd Normal Form Decomposition

Sean Chester • University of Victoria

Which are valid FD's?

- social_insurance_number → drivers_license_id
- species → phylum
- recipe_id → ingredient
- year month day → is_holiday
- postal_code → city
- v_number → gpa major birthdate

Which of the following is $\{A\}^+$

$AB \rightarrow C$

$D \rightarrow C$

$BC \rightarrow D$

$C \rightarrow A$

- ❏ {A}
- ❏ ∅
- ❏ {A, B}
- ❏ {A, B, C}
- ❏ {A, B, C, D}
- ❏ {B, C, D}

Today's Goal:

You can prevent data anomalies among a set of relations by decomposing a relation into Boyce-Codd Normal Form (BCNF) using its functional dependencies (FD's)

Outline

1. Review FD's & closures
2. Keys & superkeys
3. BCNF violations
4. The BCNF decomposition algorithm
5. Natural joins & decompositions
6. Summary / Exercise

Last class...

Deletion Anomalies

- What happens if we execute the SQL code below?

```
DELETE
FROM `Enrolments`
WHERE `v_number` = ‘V00333444’;
```

v_number	name	class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370

Deletion Anomaly: If some information is deleted (e.g., Bob drops CSC 370), we may lose the rest of the information (Bob’s name and v-number).

Data Anomalies

Redundancy: Information is unnecessarily repeated; in this case Alice's name appears twice

Deletion Anomaly: If some information is deleted (e.g., Bob drops CSC 370), we may lose the rest of the information (Bob's name and v-number).

Update Anomaly: If we change some information we may lose consistency in the data. For example, if Alice changes her name to 'Carol' we may forget to replace both instances. (It is possible to make this impossible.)

Insertion Anomaly: If we insert new tuples, we could introduce data errors, perhaps even violate the database design. Here, we gave Alice's student number to Bob, but the student number should be unique to each student.

Functional dependency: formal definition

Consider a schema $R(C_1, C_2, ..., C_k)$ and let $C = \{C_1, C_2, ..., C_k\}$ be the set of attributes

Let $A = \{A_1, A_2, ..., A_n\} \subseteq C$ and $B = \{B_1, B_2, ..., B_m\} \subseteq C$ be subsets of R's attributes.

We say that A *functionally determines B*, denoted $A \rightarrow B$, if and only if:

> For every two possible tuples of R,
> if they have the same values for *all*
> attributes in A, then they must have
> the same values for all attributes in B.

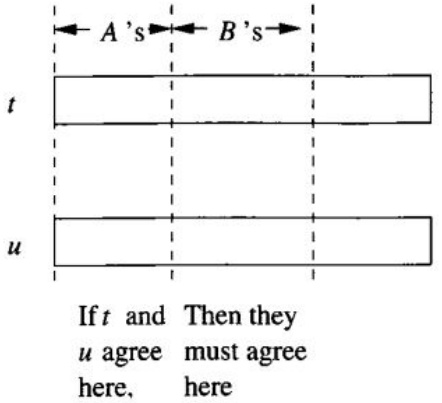


Figure 3.1: The effect of a functional dependency on two tuples.

Source: Garcia-Molina et al. (2007), §3.1.1

Functional Dependency: formal definition

We say that sin→date_joined because:

- For any two possible tuples of this relation, they cannot have the same sin and different join dates
- Intuitively, SIN is unique to each person, and a person has a constant/fixed date_joined

member_id	sin	date_joined	account_id	balance	date_opened
000111	700100200	2011-01-08	10023	$5200	2011-01-08
000111	700100200	2011-01-08	10187	$0	2011-01-08
000222	912345678	2020-03-21	10331	$100	2020-03-23
000333	700300400	2021-09-20	10855	$37910	2021-09-20
000444	700500600	2000-04-02	10001	$12500	2000-04-02
000444	700500600	2000-04-02	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.1.1

Some rules that we can apply to FD's

Because functional dependencies are logical inferences, we can apply a number of logic rules to expand our set of FD's

Transitivity
Given two FD's of the form $A \rightarrow B$ and $B \rightarrow C$, we can infer that $A \rightarrow C$

Given the FD's below:
album → artist_id
artist_id → artist_name

We can use transitivity to infer:
album → artist_name

Source: Garcia-Molina et al. (2007), §3.2.6

The *closure* of an attribute set

Using these rules, a natural question that we might ask is:

Given a set of attributes X, what is the maximum set of attributes that we can functionally determine from them?

This is called the closure of X, and is denoted X^+

Consider that you have the following FD's:
A→C
BC→D
C→*B*

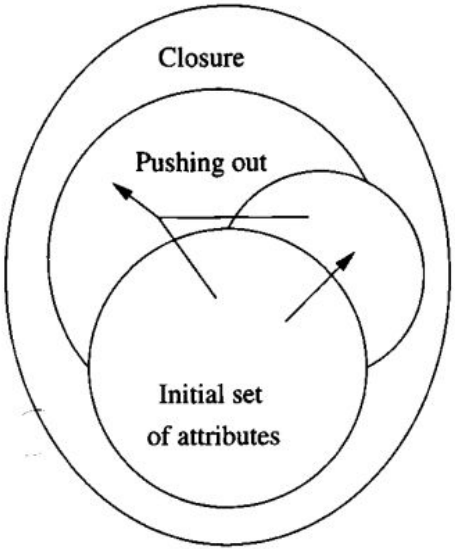


Figure 3.4: Computing the closure of a set of attributes

What is the closure of {A}, i.e., what is $\{A\}^+$?

1. Observe that A→C; so, $\{A\}^+ \supseteq \{A,C\}$
2. Observe that C→B; so, $\{A\}^+ \supseteq \{A,B,C\}$
3. Observe that BC→D; so, $\{A\}^+ \supseteq \{A,B,C,D\}$
4. At this point, we cannot exploit any more rules; so, we conclude that $\{A\}^+ = \{A,B,C,D\}$

Source: Garcia-Molina et al. (2007), §3.2.4

Keys & superkeys

A (*super*)*key* of a relation: formal definition

Let C be the set of attributes of relation R and $A \subseteq C$ be a subset of C.

A is a *superkey* if and only if (iff):

1. $\{A\}^+ = C$ (i.e., every attribute of R can be functionally determined by A)

Source: Garcia-Molina et al. (2007), §3.1.3

A (*super*)*key* of a relation: formal definition

Let C be the set of attributes of relation R and $A \subseteq C$ be a subset of C.

A is a *superkey* if and only if (iff):

1. $\{A\}^+ = C$ (i.e., every attribute of R can be functionally determined by A)

Moreover, A is a *key* if and only if (iff):

1. A is a superkey
2. There is no set $A' \subset A$ for A' is a superkey

In other words, A is a *minimal* set of attributes from which the value of every tuple of R can be functionally determined; i.e., R is a function from minimal domain A onto range C.

Source: Garcia-Molina et al. (2007), §3.1.2

A *key* of a relation

Consider a relation R(A,B,C,D) with the set of FD's that we have been using:
$A \rightarrow C$
$BC \rightarrow D$
$C \rightarrow B$

He have already computed that:

- $\{A\}^+ = \{A,B,C,D\}$
- $\{B,C\}^+ = \{B,C,D\}$
- $\{A, C\}^+ = \{A,B,C,D\}$

Therefore, we can see that:

- {A} is a *key* and a *superkey* for R
- {A,C} is a *superkey* for R but not a key because it is a superset of superkey {A}, i.e., it is not minimal
- {B,C} is neither a key nor a superkey for R, because it cannot determine A

Source: Garcia-Molina et al. (2007), §3.1.2

Example of a Full Database Schema

A relational database about movies could have the following logical schema:

Movies(title:string, year:integer, length:integer, genre:string, studioName:string, producer:integer)

MovieStar(name:string, address:string, gender:char, birthdate:date)

StarsIn(movieTitle:string, movieYear:string, starName:string)

MovieExec(name:string, address:string, certificateNum:integer, netWorth:integer)

Studio(name:string, address:string, presidentCertificateNum:integer)

What does this schema tell us about data uniqueness?

Source: Garcia-Molina et al. (2007), §2.2.8

Functional dependencies and keys

A→B, i.e., A *functionally determines* B iff every two tuples of R that match on every attribute of A must also match on every attribute of B.

Moreover, *A* is a *superkey* iff its closure is all of C.

Contrast with our example: v_number → name

Before

v_number	name	class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370

After

v_number	name
V00111222	Alice
V00333444	Bob

v_number	class
V00111222	CSC 370
V00111222	CSC 320
V00333444	CSC 370

Source: Garcia-Molina et al. (2007), §3.1

Which of the following are keys

$AB \rightarrow C$

$B \rightarrow D$

$D \rightarrow C$

$C \rightarrow A$

- ❏ {A}
- ❏ {B}
- ❏ {A, B, C, D}
- ❏ ∅
- ❏ {A, B, C}
- ❏ {B, C,}

BCNF violations

Boyce-Codd Normal Form

The goal of decomposition is to obtain a set of relations that are all in some *normal form*.

Boyce-Codd Normal Form (BCNF):

A relation **R** with attribute set C is in BCNF iff:

For every non-trivial functional dependency on R, $\{A_1, A_2, \ldots, A_n\} \rightarrow \{B_1, B_2, \ldots, B_m\}$,
$\{A_1, A_2, \ldots, A_n\}$ is a *superkey* of R, i.e., $A^+ = C$.

Stated informally, no relation has a functional dependency in which the antecedent does not determine every other attribute.

*Note that this requires a *superkey*, not necessarily a key.

Source: Garcia-Molina et al. (2007), §3.3.3

Checking that our example is in BCNF

We have one functional dependency: v_number → name

Before decomposition, we see that there are attributes (namely *class*) that are not determined by the antecedent (namely *v_number*) of this FD

After decomposition, the first relation no longer contains *class*, so v_number is a (super)key. The FD does not apply to the second relation, because the second relation does not have the attribute *name*

Before

v_number	name	class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370

After

v_number	name
V00111222	Alice
V00333444	Bob

v_number	class
V00111222	CSC 370
V00111222	CSC 320
V00333444	CSC 370

Source: Garcia-Molina et al. (2007), §3.3.3

Revisiting our Bank Accounts example:

- Is this in BCNF? Does it contain any BCNF violations?

member_id	sin	date_joined	account_id	balance	date_opened
000111	700100200	2011-01-08	10023	$5200	2011-01-08
000111	700100200	2011-01-08	10187	$0	2011-01-08
000222	912345678	2020-03-21	10331	$100	2020-03-23
000333	700300400	2021-09-20	10855	$37910	2021-09-20
000444	700500600	2000-04-02	10001	$12500	2000-04-02
000444	700500600	2000-04-02	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.3.4

BCNF Decomposition

What is “Decomposition”?

Consider that you have a relation **R** with attributes $\mathbf{A=\{A_1, A_2, ..., A_n\}}$

A *decomposition* of **R** splits it into two relations:

S with attributes $\mathbf{B=\{B_1, B_2, ..., B_m\}}$ and
T with attributes $\mathbf{C=\{C_1, C_2, ..., C_k\}}$.

Such that:

- $\mathbf{A = B \cup C}$
- $\mathbf{S = \pi_{(B1, B2, ..., Bm)}(R)}$
- $\mathbf{T = \pi_{(C1, C2, ..., Cm)}(R)}$

Why would we do that?

Source: Garcia-Molina et al. (2007), §3.3.2

BCNF Decomposition Algorithm

What is our systematic process (i.e., algorithm) to arrive at a decomposition in BCNF?

BCNF_Decomp(R_0, F_0)
Input: Relation R_0 with attribute set C_0 and functional dependencies F_0
Output: A decomposition of R_0 in which all relations are in BCNF

1. If R_0 is in BCNF, return R_0
2. Select a BCNF violation (an FD for which the antecedent is not a superkey), $X \rightarrow Y$
3. Compute the closure X^+
4. Let $R_1 := X^+$
5. Let $R_2 := (C \setminus X^+) \cup X$
6. Project F_0 to get functional dependencies for R_1 and R_2 denoted F_1 and F_2, respectively
7. Return **BCNF_Decomp**(R_1, F_1) $\cup$ **BCNF_Decomp**(R_2, F_2)

Source: Garcia-Molina et al. (2007), §3.3.4

Small Decomposition Example 1

Let's decompose this together:

Schema:	Functional Dependencies
R(A, B, C)	{A}→{B}
	{B}→{C}

How do we start?

Source: Garcia-Molina et al. (2007), §3.3.4

Small Decomposition Example 1

Let's decompose this together:

Schema:	Functional Dependencies
R(A, B, C)	{A}→{B}
	{B}→{C}

How do we start?

1. Check if {A} and {B} are superkeys, i.e., does $\{A\}^+=\{A,B,C\}$ and $\{B\}^+=\{A,B,C\}$

Source: Garcia-Molina et al. (2007), §3.3.4

Small Decomposition Example 1

Let's decompose this together:

Schema:	Functional Dependencies
R(A, B, C)	{A}→{B}
	{B}→{C}

How do we start?

1. Check if {A} and {B} are superkeys, i.e., does $\{A\}^+=\{A,B,C\}$ and $\{B\}^+=\{A,B,C\}$

   NO! A cannot be determined from B. Thus the FD {B}→{C} is a *BCNF violation*
2. Calculate sets $R_1 = \{B\}^+$ and $R_2 = \{A,B,C\} \setminus \{B\}^+ \cup \{B\}$
3. Project the FD's onto each relation to get $F_1 = \{\{A\}\rightarrow\{B\}\}$ and $F_2 = \{\{B\}\rightarrow\{C\}\}$

Final BCNF Decomposition: $R_1(A,B)$ and $R_2(B,C)$

Source: Garcia-Molina et al. (2007), §3.3.4

Small Decomposition Example 2

Let's decompose this together:

Schema:
R(A, B, C, D, E)

Functional Dependencies
{A,B}→{C}
{C}→{D}
{D}→{E}

Source: Garcia-Molina et al. (2007), §3.3.4

Small Decomposition Example 2

Let's decompose this together:

Schema: **R(A, B, C, D, E)**

Functional Dependencies
{A,B}→{C}
{C}→{D}
{D}→{E}

1. We see that {A,B} is a key, but neither {C} nor {D} are. Thus we have *two* BCNF violations.
2. Let's decompose using {C}→{D}:
   a. Calculate sets $R_1 = \{C\}^+$ and $R_2 = \{A,B,C,D,E\} \setminus \{C\}^+ \cup \{C\}$
   b. Thus we get $R_1 = \{C,D,E\}$ and $R_2 = \{A,B,C,D,E\} \setminus \{C,D,E\} \cup \{C\} = \{A,B,C\}$
   c. Project the FD's onto each relation to get $F_1 = \{\{C\}\rightarrow\{D\}, \{D\}\rightarrow\{E\}\}$ and $F_2 = \{\{A,B\}\rightarrow\{C\}\}$
3. But we *still* have a BCNF violations on R_1, namely {D}→{E}. So we recursively decompose:
   a. Calculate sets $R_{11} = \{D\}^+ = \{D,E\}$ and $R_{12} = \{C,D,E\} \setminus \{D\}^+ \cup \{D\} = \{C,D\}$
   b. Project the FD's onto each relation to get $F_{11} = \{\{D\}\rightarrow\{E\}\}$ and $F_{12} = \{\{C\}\rightarrow\{D\}\}$

Final BCNF Decomposition: $R_{11}(C,D)$ and $R_{12}(D,E)$ and $R_2(A,B,C)$

Source: Garcia-Molina et al. (2007), §3.3.4

Revisiting our Bank Accounts example:

We observe FD's: account_id → balance date_opened member_id AND member_id → sin date_joined
The second FD is a BCNF violation. Indeed, somebody could add a new account and mistype the SIN

member_id	sin	date_joined	account_id	balance	date_opened
000111	700100200	2011-01-08	10023	$5200	2011-01-08
000111	700100200	2011-01-08	10187	$0	2011-01-08
000222	912345678	2020-03-21	10331	$100	2020-03-23
000333	700300400	2021-09-20	10855	$37910	2021-09-20
000444	700500600	2000-04-02	10001	$12500	2000-04-02
000444	700500600	2000-04-02	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.3.4

Revisiting our Bank Accounts example:

We observe FD's: account_id → balance date_opened member_id AND member_id → sin date_joined
The second FD is a BCNF violation. Indeed, somebody could add a new account and mistype the SIN

Let's do a decomposition to eliminate the BCNF violation on the second FD:

R1 = {member_id}+ = {member_id, sin, date_joined}
R2 = {account_id,balance,date_opened, member_id, sin, date_joined}\{member_id, sin, date_joined}∪{member_id}

R1 = {member_id, sin, date_joined}
R2 = {account_id, balance, date_opened, member_id}

So, we end up with the new set of relations:
Member(member_id, sin, date_joined)
Account(account_id, balance, date_opened, member_id)

Source: Garcia-Molina et al. (2007), §3.3.4

Revisiting our Bank Accounts example:

Our decomposed relation instances now look like this.

- Compare what happens now if we delete acc_id 10331 or add a new account for member 000333
- Consider also what happens if we want to add somebody without an account yet or update a SIN
- We only “duplicate” member_id, not all info about the member. This much is unavoidable.

member_id	sin	date_joined
000111	700100200	2011-01-08
000222	912345678	2020-03-21
000333	700300400	2021-09-20
000444	700500600	2000-04-02

member_id	acc_id	bal	opened
000111	10023	$5200	2011-01-08
000111	10187	$0	2011-01-08
000222	10331	$100	2020-03-23
000333	10855	$37910	2021-09-20
000444	10001	$12500	2000-04-02
000444	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.3.4

Large Decomposition Example

Let's decompose this together:

Schema:
R(A, B, C, D, E)

Functional Dependencies
$A \rightarrow B$
$B \rightarrow D$
$CD \rightarrow E$

Source: Garcia-Molina et al. (2007), §3.3.4

Try this one on your own…

Decompose this relation into BCNF:

Schema:	Functional Dependencies
R(A, B, C)	$A \rightarrow B$
	$B \rightarrow A$

Source: Garcia-Molina et al. (2007), §3.3.4

Natural joins & decompositions

Lossless Decompositions

- What if we want our original data back?

member_id	sin	date_joined
000111	700100200	2011-01-08
000222	912345678	2020-03-21
000333	700300400	2021-09-20
000444	700500600	2000-04-02

member_id	acc_id	bal	opened
000111	10023	$5200	2011-01-08
000111	10187	$0	2011-01-08
000222	10331	$100	2020-03-23
000333	10855	$37910	2021-09-20
000444	10001	$12500	2000-04-02
000444	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.3.4

Lossless Decompositions

- What if we want our original data back?
  - Members ⋈ Accounts

member_id	sin	date_joined
000111	700100200	2011-01-08
000222	912345678	2020-03-21
000333	700300400	2021-09-20
000444	700500600	2000-04-02

member_id	acc_id	bal	opened
000111	10023	$5200	2011-01-08
000111	10187	$0	2011-01-08
000222	10331	$100	2020-03-23
000333	10855	$37910	2021-09-20
000444	10001	$12500	2000-04-02
000444	10002	$2500	2000-04-02

Source: Garcia-Molina et al. (2007), §3.3.4

Lossless Decompositions

- What if we want our original data back?
  - Members ⋈ Accounts

member_id	sin	date_joined
000111	700100200	2011-01-08
000222	912345678	2020-03-21
000333	700300400	2021-09-20
000444	700500600	2000-04-02

member_id	acc_id	bal	opened
000111	10023	$5200	2011-01-08
000111	10187	$0	2011-01-08
000222	10331	$100	2020-03-23
000333	10855	$37910	2021-09-20
000444	10001	$12500	2000-04-02
000444	10002	$2500	2000-04-02

```
SELECT *
FROM Members
NATURAL JOIN Accounts;
```

Source: Garcia-Molina et al. (2007), §3.3.4

Summary / Exercise

Concept summary

- Anomalies could occur because FD's in a relation did not involve keys
- Student number should determine name, but if there are multiple such tuples, the constraint can be broken
- We call such cases a "BCNF violation" and use decomposition to eliminate them
- The theory nicely leads to restoring the original data with natural joins

Before

v_number	name	class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370

After

v_number	name
V00111222	Alice
V00333444	Bob

v_number	class
V00111222	CSC 370
V00111222	CSC 320
V00333444	CSC 370

Source: Garcia-Molina et al. (2007), §3.1

Large Decomposition Example

Schema:
R(A, B, C, D, E)

Functional Dependencies
$A \rightarrow B$
$B \rightarrow D$
$CD \rightarrow E$

a) What are the keys?
b) Is it in BCNF?
c) Can you decompose it into BCNF?

Source: Garcia-Molina et al. (2007), §3.3.4